

Your Prompt Is Only Half the Prompt: Silent Chat-Template Drift in Open-Weight Model Repositories

Simarjot Khanna
Independent Researcher
Canada
simarkhanna76@gmail.com

Abstract

PromptOps treats prompts as code: versioned, reviewed, and defended by a regression suite. The prompt a model actually reads is not the prompt a team writes. It is first rewritten by the model's *chat template*, an executable Jinja program shipped inside the model repository, edited in place, with no version of its own. How often this hidden dependency changes, and with what behavioural consequence, has not been measured. We reconstruct the default-branch template history of 400 of the most-downloaded text-generation repositories on the Hugging Face Hub. A renderer validated byte-for-byte against the reference implementation decides whether each edit changes what the model is shown, and we run 5 local checkpoints under their own repositories' historical templates. 43.8% of the repositories changed their template after first publishing one, and 80.6% of the change events moved no weight file, invisible to a team that pinned the model and verified its weights; 50.2% of the decidable edits change what the model is shown. Much of the raw rate is launch-week churn, and we report the settled figure beside it: 8.5% of the frame saw such an edit land more than 30 days after its first template. In a one-repository case study, an edit that dropped the system role halved output-contract compliance from 90.0% to 45.0% until a later edit largely restored it. `TEMPLATELOCK` pins what a template renders rather than what it says, separating the edits that matter from the ones that do not at 9.2 ms per model. Until prompt pipelines version the template, pinning the prompt and the model does not pin the model's input.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; **Software configuration management and version control systems**.

Keywords

PromptOps, chat templates, prompt versioning, drift detection, open-weight models, reproducibility

ACM Reference Format:

Simarjot Khanna. 2026. Your Prompt Is Only Half the Prompt: Silent Chat-Template Drift in Open-Weight Model Repositories. In *Proceedings of the 1st International Workshop on PromptOps and Vibe Coding (POVC '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3843779.3844636>



This work is licensed under a Creative Commons Attribution 4.0 International License. POVC '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2991-1/2026/10
<https://doi.org/10.1145/3843779.3844636>

1 Introduction

PromptOps asks software teams to treat prompts as code: version them, review them, put them under test, and gate releases on a prompt regression suite. The premise is that if the prompt is pinned and the model is pinned, the system is pinned.

It is not. Between the prompt a team writes and the tokens a model reads sits a second artifact that neither pin covers. Open-weight chat models ship a *chat template*: a Jinja program, stored inside the model repository, that rewrites a list of messages into the exact string the model is served. It decides where the system message goes, or whether it survives at all; it decides which control tokens delimit a turn; it decides what is appended to invite the assistant to speak. Change it and the model sees different input, even though the prompt is byte-identical and the weights have not moved.

That artifact is edited in place. It lives on a repository's default branch with no version of its own, so a team that pulls a model by name gets whichever template happened to be committed most recently. There is no semantic version to pin, no changelog convention, and no equivalent of the package-manager machinery that at least records when a dependency moved. The npm ecosystem has version numbers and breaking changes still reach clients through releases that were supposed to be backward compatible [13]. Chat templates do not have the version numbers.

The hazard of a mismatched template was named when the mechanism shipped [1]; how often the template itself moves, and to what effect, has not been measured. That is the gap this paper closes, with three research questions. **RQ1**: how often does the chat template of a popular open-weight repository change after release, and how much of that change is invisible to a team that pins the model and verifies its weights? **RQ2**: when the template changes, does the model's behaviour follow? **RQ3**: can a check tell the edits that change the model's input from the edits that do not, cheaply enough to run in every build? To answer them we reconstruct the default-branch history of every chat template in a frame of 400 of the most-downloaded text-generation repositories on the Hugging Face Hub. We decide which edits actually change what a model is shown by *rendering* rather than by reading the Jinja source, and we then hold a checkpoint and a conversation fixed while varying only the template.¹

The question matters most for exactly the practice this workshop is named after. A developer prompting a model and judging the result by running it [3] has no way to separate a model that has become worse from a prompt wrapper that changed underneath. A grey-literature review of 101 practitioner sources finds that quality

¹Artifact: <https://github.com/s1mar/chat-template-drift>

assurance in this style of work is frequently skipped altogether [4], which leaves the template free to move without anyone noticing.

Contributions.

- The first empirical measurement of chat-template drift (RQ1). In a frame of 400 repositories, 43.8% changed their chat template after first publishing one, and 80.6% of change events occurred in an interval where no model-weight file moved.
- A rendering-based test that separates cosmetic edits from behavioural ones (RQ1), validated byte-for-byte against the reference implementation. It shows that 50.2% of decidable change events alter what the model is shown, and that textual diff size does not predict which.
- An execution-grounded study over 5 checkpoints run under their own repositories' historical templates, with a replication control that establishes the noise floor (RQ2). Its strongest effect, an edit that halves output-contract compliance, comes from one repository and is scoped as a case study throughout.
- `TEMPLATELOCK`, a check that pins a template by what it renders rather than by its text (RQ3). Replayed over the frame it raises 137 alerts where a plain hash raises 289, and it costs 9.2 ms per model.

2 The chat template is executable code

A chat model is trained on a specific serialisation of dialogue. Serving it a differently serialised conversation is out of distribution, so the serialisation is shipped with the model. In the Hugging Face ecosystem it is the `chat_template` field of `tokenizer_config.json`, or a sibling `chat_template.jinja`, and it is applied by `apply_chat_template` [5].

Three properties make it a supply-chain artifact rather than a configuration detail. It is *executable*, being a Jinja program with conditionals and loops rather than a format string. It is *invisible at the call site*: application code passes a message list and never sees the rendered result. And it is *unversioned*: it has no identity separate from the repository revision that carries it, so pinning it means pinning a commit, which the common tooling paths do not do by default.

The failure this creates is specific. A prompt suite that asserts properties of a model's reply will happily re-run after a template edit, exercise a different serialisation, and report whatever it finds as a property of the prompt.

3 Study design

The design, the frame, the definitions and a falsification criterion were fixed in advance and are part of the released artifact. Each element of the design serves one research question: the frame and the change taxonomy serve RQ1, the execution arms and their controls serve RQ2, and the replayed check serves RQ3; Sections 4, 5 and 6 answer them in that order.

3.1 Frame

We list Hub models with `pipeline_tag = text-generation` in descending all-time downloads as recorded on 2026-07-30, the day the frame was drawn, and take them in order. A repository is included if an anonymous clone succeeds and at least one default-branch

revision carries a chat template. The frame is the first 400 repositories that qualify, drawn from 1000 candidates. Of those candidates, 618 qualified, 205 never carried a template-bearing file, 136 carried the file but never a template in it, and 41 were gated or absent. No candidate is unaccounted for. The 218 repositories that qualified beyond rank 400 are held out and reported separately; they are never merged into the frame.

History is reconstructed with one blob-filtered, single-branch clone per repository (the collector in the released package), after which the whole commit graph is read locally. Model weights are Git LFS objects and are never fetched.

3.2 What counts as a change, and as a silent one

A *template revision* is a default-branch commit at which the extracted template string differs from the previous one. A repository has *drifted* if it exhibits at least two distinct templates, which is to say the template you get depends on when you pulled.

An interval between consecutive template revisions is *weight-silent* if no tracked model-artifact file changed in it. This is the case that defeats ordinary diligence: a team that pinned the model by name and verified its weights would have seen nothing at all.

3.3 Cosmetic or behavioural, decided by rendering

Reading a Jinja diff is a poor way to decide whether an edit matters, because a large reformatting can be inert and a four-character edit can drop the system message. We decide it by rendering. Six probe conversations, fixed in advance, are rendered through both templates: a single user turn; system plus user; a four-message multi-turn exchange; a turn with the generation prompt requested and one without; and a system message carrying an output-contract instruction. A pair is *behavioural* if any probe renders differently, or if one template renders a probe that the other refuses. A pair is *cosmetic* if every probe is byte-identical, however large the textual diff. A pair where *neither* template renders a probe is *undecidable* for that probe, and a pair with no decidable probe is excluded from the split and counted in its own right.

That third category is not fastidiousness. An early version of this analysis folded both-failed into "identical", a missing import made every render fail, and the pipeline reported zero behavioural changes out of 289 with no error anywhere. A category that can absorb total instrument failure and still present as a clean null must not exist. The 16 undecidable events that do occur have three concrete causes, and in every one both members of the pair fail. Some templates are written against a nonstandard Jinja extension tag (`generation`) that the sandboxed Jinja environment pinned for this study rejects. Others fail to parse at either revision: in one repository a stretch of consecutive revisions carries the same parse error, commits titled as fixes ("fix typo with missing closing curly bracket") still left the template unparseable, and by definition only a revision that renders again can make a pair decidable. A few templates include a companion template file that our loader-free sandbox cannot resolve. The first and third causes are limits of our renderer; the second is what a repair in progress looks like from outside the repository. All 16 are excluded by the rule above: a pair with no decidable probe is not scored.

Renderer validation. Every behavioural claim rests on our renderer agreeing with the one practitioners actually run. Before any measurement was believed, our renderer was required to equal the reference `apply_chat_template` byte for byte on every probe of every repository in a fixed list of eight reference checkpoints. It does, on the 7 whose tokenizers load in our pinned environment; the eighth needs a tokenizer backend outside it. A renderer that silently differed would manufacture divergence, which is the direction that flatters this paper.

Checking current templates leaves a gap, because the study renders *historical* ones. We close it rather than bound it: the Hub serves any revision, so for each of the 16 templates used as an execution arm below (one arm is one historical template a checkpoint is run under) we load that revision’s own tokenizer and compare its `apply_chat_template` against ours on the same probes. 0 of the 16 mismatch. The same check confirms that the template we extracted from the commit graph is the one the reference implementation loads at that revision, so both the extraction and the rendering are validated on exactly the inputs the results depend on.

3.4 Execution: same weights, same conversation, different template

Mining shows that renderings change. Whether the model cares is a different question, and the only way to answer it is to run the model.

We use 5 local checkpoints, each paired with the Hub repository whose template history it is served under: Qwen2.5-7B-Instruct, Phi-3-mini-4k-instruct, deepseek-coder-6.7b-instruct, Mistral-7B-Instruct-v0.2, and Meta-Llama-3-8B-Instruct. They were selected as widely deployed open-weight instruction-tuned models with reconstructable template histories at the time of the study; every RQ2 comparison is within-checkpoint, so checkpoint generation is not a confound of any comparison. For each checkpoint we take its repository’s render-distinct historical templates, render an identical repair conversation through each, and send the rendered string to the same local weights through a local Ollama endpoint whose raw mode bypasses the server’s own templating. The conversation, the decoding parameters, the seed, the stop tokens and the weights are identical in every arm. The rendered prompt string is the only thing that differs.

The task is single-function program repair over a seeded sample of 60 tasks from the code-repair split of HumanEvalPack [8], graded by executing the task’s own test. An episode is one task sent once under one template arm; decoding is greedy (temperature zero) with a 1536-token generation cap, on the quantised builds named, with their digests, in the artifact. Identical rendered prompts are served from a generation cache, so two arms that render a conversation identically receive the same stored reply; the replication arms are keyed separately, so a floor is always a fresh generation. The cap truncates 4 replies, all retained and scored. We record whether the returned program passes, whether the reply obeyed the stated output contract, and a hash of the reply. The contract is the system message’s closing instruction, “Reply with exactly one fenced python code block and no other text”; a reply complies if it contains exactly one fenced block and that block defines the task’s entry point.

The noise floor. Greedy decoding on a GPU is not perfectly reproducible, so a divergence rate means nothing without knowing what zero looks like. Each checkpoint therefore runs its first template a second time, byte-identical prompt and settings, and the divergence between those two runs is the floor against which every cross-template comparison is read. We report each checkpoint’s replication discordance next to its cross-template discordance, so a difference no larger than the checkpoint’s own noise cannot be mistaken for an effect.

Does the transport preserve the template? Sending an already-rendered string risks the server re-tokenising the template’s control markers as ordinary text, which would mean we were measuring our own plumbing. We test it directly rather than assume it. Each checkpoint’s own marker is submitted twice, once intact and once with its characters separated, and on all 4 checkpoints checked the separated characters cost at least 1.7 times as many tokens as the intact marker, so no marker is being shattered into its characters. On Qwen2.5-7B-Instruct and on Phi-3-mini-4k-instruct, the checkpoint carrying the behavioural result below, the server’s token count for the marker equals the reference tokenizer’s exactly. The two other checkpoints differ from the reference by one token on a marker measured in isolation, Mistral-7B-Instruct-v0.2 and Meta-Llama-3-8B-Instruct, and neither bears on a claim: Mistral-7B-Instruct-v0.2’s effect is a rendering exception raised before a single token exists, and Meta-Llama-3-8B-Instruct yields no informative comparison at all. deepseek-coder-6.7b-instruct, whose comparisons all fail to reject, uses no bracketed marker and is not part of this probe. This is a bound rather than a proof of identical token identifiers, which the server does not expose.

3.5 Analysis

All comparisons are paired over tasks, since every arm sees every task. Paired binary outcomes use McNemar’s exact test on the discordant pairs; intervals are percentile bootstraps over tasks with 10,000 resamples. Multiplicity is corrected with Holm over the family of every informative within-checkpoint pair, separately for each outcome. The family contains both the consecutive transitions and the first-versus-last comparisons, so both kinds are corrected and reported, not only the one that suits the argument.

4 RQ1: how often, and how silently

Table 1 reports the frame. Of the 400 repositories in it, 175 (43.8%) changed their chat template at least once after first publishing one, across 289 change events. The largest number of distinct templates in a single repository is 9.

Rendering splits those events almost exactly in half. Of the 273 decidable events, 137 (50.2%) change what the model is shown and 136 render identically. The remaining 16 of the 289 events are undecidable. At the repository level, 103 repositories (25.8% of the frame, 58.9% of those that drifted) carry at least one behavioural change.

233 of the 289 events (80.6%) are weight-silent, and 76.7% of repositories with a behavioural change have one in a weight-silent interval. The prompt wrapper moved while the model, by every check a careful team would run, did not.

Table 1: Chat-template drift in the frame. *Behavioural* means the edit changes what the model is shown, decided by rendering six fixed probe conversations through both templates. *Weight-silent* means no model-weight file changed in the interval, so a consumer who pinned the model and verified its weights would have seen nothing.

	count	share
Repositories in frame	400	
drifted (template changed at least once)	175	43.8%
with a behavioural change	103	25.8%
behavioural and settled (> 30 days)	34	8.5%
behavioural in a weight-silent interval	79	76.7% of beh.
Template-change events	289	
decidable	273	
behavioural	137	50.2% of dec.
cosmetic (renders identically)	136	
undecidable (neither renders)	16	
weight-silent	233	80.6%

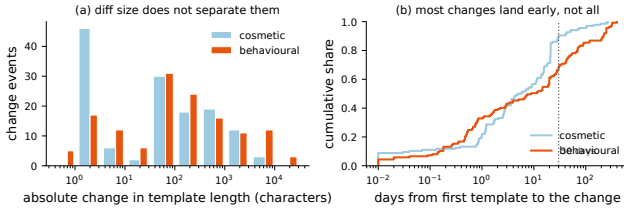


Figure 1: (a) The textual size of a template edit does not separate the edits that change what the model sees from the ones that do not; the distributions overlap across four orders of magnitude. (b) When the changes land, relative to a repository’s first template. Most arrive within days, which is why the paper reports a separate settled figure, but the tail extends well past a month.

Most drift is early, and we say so plainly. The median time from a repository’s first template to its first change is 1.5 days. Much of the headline 43.8% is therefore launch-week churn, a maintainer settling a template shortly after release. That is a weaker claim than it first appears and we do not lean on it. The conservative figure is drift that is both behavioural and settled: 34 repositories, 8.5% of the frame, changed what the model is shown more than 30 days after first publishing a template. Early churn is not harmless either, since anyone who pulled during that window is affected and the Hub does not tell them, but it is a different and smaller claim.

Beyond the frame, of the 218 qualifying repositories at ranks past 400, 34.4% drifted and 17.4% of that whole set carries a behavioural change. Both denominators are the out-of-frame set, matching the two frame-level figures above rather than the share-of-drifted figure. The phenomenon persists further down the popularity ranking at a lower rate. The headline rates are measured on the most-pulled repositories on the Hub, not on a neglected tail.

Diff size does not tell you which is which. Textual diff size is close to useless as a proxy: as a discriminator between behavioural and

Table 2: Same checkpoint, same conversation, same seed; only the historical chat template differs. Checkpoint names drop their Instruct token for width; full identifiers are in Section 3.4. *Floor* is the response divergence between two runs of the *same* template, a single re-run of each checkpoint’s first template: the serving stack’s reproducibility noise for that checkpoint. *Resolution* is the share of 60 repair tasks whose returned program passes its test; *contract* is the share of replies obeying the stated output contract. [†] this template refuses the conversation outright: it raises on a system message, so all 60 tasks fail before the model is reached.

checkpoint	floor	template	resolved	contract
Mistral-7B-v0.2	0.0%	2023-12-11	0.0% [†]	0.0%
		2024-07-03	28.3%	100.0%
Phi-3-mini-4k	0.0%	2024-04-22	21.7%	90.0%
		2024-04-29	10.0%	45.0%
		2024-07-01	23.3%	88.3%
deepseek-coder-6.7b	90.0%	2023-11-01	40.0%	98.3%
		2023-11-01	43.3%	98.3%
		2023-11-04	43.3%	96.7%
		2023-11-28	45.0%	95.0%
		2023-11-29	45.0%	98.3%
		2023-12-13	45.0%	98.3%
Qwen2.5-7B	15.0%	2024-09-16	28.3%	40.0%
		2024-09-18	28.3%	40.0%
Meta-Llama-3-8B	26.7%	2024-04-18	43.3%	100.0%
		2024-04-19	43.3%	100.0%
		2024-04-30	43.3%	100.0%

cosmetic edits, the absolute change in template length achieves an AUC of 0.57, barely above chance. The median absolute change in template length is 52 characters for cosmetic edits and 86 for behavioural ones, and the distributions overlap heavily: the largest purely cosmetic edit in the frame changes 4403 characters and alters nothing the model sees, while 5 behavioural edits change the template’s length by zero. A reviewer skimming a diff for size will misclassify both tails, which is the argument for deciding this by rendering. Figure 1(a) shows the two distributions on top of each other.

5 RQ2: does the model follow?

RQ1 established that renderings change. RQ2 asks whether the model’s behaviour follows, which only running the model can answer.

Table 2 reports every checkpoint, its replication floor, and the two outcomes under each of its historical templates. Across 5 checkpoints and 1260 episodes, 5 of the template pairs turned out to render *this* conversation identically despite differing on the probe set, because the edit only touches a branch this conversation does not take. Those pairs vary nothing and are excluded: an identical input producing an identical output is not evidence that the edit was harmless. That leaves 18 informative comparisons, and both outcome families are Holm-corrected across all 18.

The floor first, because it disqualifies the obvious metric. Running the same template twice, byte-identical prompt and settings, changes the reply on 0.0% of tasks for Mistral-7B-Instruct-v0.2 and Phi-3-mini-4k-instruct but on 90.0% for deepseek-coder-6.7b-instruct. Response divergence is therefore useless as a drift signal on its own, since on deepseek-coder-6.7b-instruct it is near-saturated before any template changes at all. What survives replication is the executed outcome, and that is what we test.

A template edit can make the request impossible. Mistral-7B-Instruct-v0.2's original template raises on a system message. All 60 tasks fail before the model is reached, giving resolution 0.0% and contract compliance 0.0%. A later edit to the same repository, on the same weights, accepts system messages: resolution 28.3% and contract 100.0%. This is the one resolution comparison that survives Holm across all 18 tests ($p_{\text{Holm}} = 0.0003$; the contract swing also survives, at $p_{\text{Holm}} < 0.0001$), and it is categorical rather than statistical. A pipeline pinned to the earlier revision cannot send a system prompt at all, and one written against the later revision fails completely if it pulls the earlier one.

A template edit can halve what a prompt suite asserts, and a later edit can restore it. This is the strongest execution-grounded effect we observe, and it is a one-repository case study: it establishes that the failure occurs, not how often. On Phi-3-mini-4k-instruct, contract compliance runs 90.0%, then 45.0%, then 88.3% across the repository's three templates, while resolution runs 21.7%, 10.0%, 23.3%. Both contract transitions survive correction, down at $p_{\text{Holm}} < 0.0001$ and back up at $p_{\text{Holm}} < 0.0001$. The size of the swing is the operational point rather than the p-value: compliance halves at one edit and is restored at the next, a movement far outside the checkpoint's own replication floor. The middle template is the commit whose own message describes replacing the system role with the user role, which is what a halved compliance rate on a system-carried output contract looks like from the outside. The resolution swing over the same two transitions is large but does not survive correction across 18 tests ($p_{\text{Holm}} = 0.250$ and 0.133); with only 7 and 8 discordant tasks the smallest p-value these exact tests can produce cannot clear a family of 18, so the design is underpowered for these transitions and we report them as suggestive rather than as established or refuted.

This is where a then-versus-now check fails. Comparing Phi-3-mini-4k-instruct's first template with its last gives $p_{\text{Holm}} = 1.00$ on resolution and a contract rate back to roughly where it started. The repository broke and then repaired itself. A check that compares only the current template against the one it was pinned at sees nothing, while what a consumer lived through was the window in between.

Drift is often harmless. deepseek-coder-6.7b-instruct ships 6 distinct templates and none of the 14 informative comparisons between them moves resolution detectably, the smallest being $p_{\text{Holm}} = 1.00$, with resolution staying between 40.0% and 45.0%. Of the 18 informative comparisons, 1 survives correction on resolution and 3 on contract compliance. The claim is not that template edits usually break things. It is that some do, that the ones that do cannot be told from the ones that do not without running the model, and that

nothing in the tooling available when this study was run tells a consumer which kind has just landed.

Two robustness checks, because both cut against us if they fail. First, the excluded uninformative pairs are put back and everything is re-corrected over all 23 pairs. This is the *more* conservative direction, since each restored pair contributes $p = 1$ by construction and enlarges the family, and the counts are unchanged: 1 on resolution and 3 on contract. Second, each effect is read against its own checkpoint's replication discordance. Phi-3-mini-4k-instruct and Mistral-7B-Instruct-v0.2 show 0 and 0 discordant tasks when a template is replayed against itself, against 7 and 8 for the two Phi-3-mini-4k-instruct transitions and 17 for Mistral-7B-Instruct-v0.2, so those effects sit well outside their own noise. Mistral-7B-Instruct-v0.2's floor is trivially clean, since its replicated arm fails to render every task; its effect rests on the categorical contrast above, not on that floor. Qwen2.5-7B-Instruct is the cautionary case in the other direction: replaying an identical prompt moves its resolution enough that the replication arm would itself test significant ($p = 0.031$) against a zero-discordance null. Any single comparison on a checkpoint like that should be disbelieved, which is precisely why the floor is measured rather than assumed.

5.1 What actually changed

Neither effect above needs a subtle explanation. Both are visible in the template source, and we give them here because the mechanism is what makes the result actionable rather than anecdotal.

Phi-3-mini-4k-instruct's second template deletes one branch of the role dispatch. The earlier template handles three roles:

```
{% if (message['role'] == 'system') %}...
{% elif (message['role'] == 'user') %}...
{% elif message['role'] == 'assistant' %}...
```

The later one handles two, and the system arm is simply gone. A system message therefore matches no branch and is emitted nowhere: it is discarded in silence, with no error and no warning. Our system message is where the output contract lives, which is why contract compliance falls to 45.0% and recovers when a subsequent edit restores the role.

Mistral-7B-Instruct-v0.2's original template is stricter and fails louder, though only at the point of rendering:

```
{% else %}{{ raise_exception(
  'Only user and assistant roles are supported!') }}
```

Any system message raises before a single token reaches the model. The later template accepts one. So the same repository, at two revisions, is the difference between a system prompt being impossible and a system prompt being ordinary, with no version number anywhere between them.

Both are one-branch edits. Neither changes a weight. Neither announces itself.

5.2 A control that could have refuted us

Both effects above involve the system role, so the mechanism implies a prediction that can fail: move the output contract out of the system message and into the user turn, and the damage should disappear. That prediction is the falsification criterion fixed in the design. We recorded it before running the control, which repeats

the study, the same tasks, decoding and grading, with a system-free conversation, on the two checkpoints that produced an effect, 420 more episodes. The control measures its own replication floors, 0.0% and 3.3% response divergence; the latter differs from the main run's 0.0% for the same checkpoint, which is why floors are measured per run rather than assumed.

Both predictions hold. Mistral-7B-Instruct-v0.2's earlier template raises on a system message and nothing else, so with no system message it renders every task: render errors fall from 60 to 0, and both of its templates now resolve 28.3% with 100.0% contract compliance. The gap is gone. On Phi-3-mini-4k-instruct, contract compliance becomes 95.0%, 95.0% and 93.3% across the same three templates, so the collapse at the middle template is gone as well. Of the 3 informative comparisons in the control, 0 survive correction on resolution and 0 on contract compliance.

One further check falls out of the design rather than being asked for. With no system message, Phi-3-mini-4k-instruct's first two templates render this conversation *identically*, so the comparison becomes uninformative by the same rule used throughout. That is deductive, not statistical: the two templates differ only in the system arm of their role dispatch, shown in Section 5.1, so removing the system role removes the entire difference between them.

Is this a role effect or merely a position effect? Moving an instruction also moves it, so the control changes two things at once and the recovery could in principle be the contract landing somewhere more effective rather than landing at all. Two things rule that out.

First, under the templates that fail there is no instruction to reposition. Searching the rendered prompt for the contract sentence finds it under Phi-3-mini-4k-instruct's first and third templates and *not* under its second: the deleted system arm means the text is emitted nowhere. Mistral-7B-Instruct-v0.2's earlier template does not render at all. The contrast is presence against absence, not one position against another.

Second, if position mattered independently, moving the contract would shift outcomes on templates that handle the system role correctly (operationally, among the two checkpoints re-run system-free, those whose main-run arm renders every task and keeps the contract satisfied on most replies). The shifts there are small: across the 3 such templates here, moving the contract into the user turn changes resolution by at most 1.7 points and contract compliance by at most 5.0; Mistral-7B-Instruct-v0.2's later template is unchanged on both, at 28.3% and 100.0%. The recovery appears only where the template mishandles the role, which is what a role effect predicts and a position effect does not.

The mechanism is therefore not “the template changed” in general. It is “the template changed how it handles the role your contract lives in”. That is a smaller claim than the one we could have made from the main run alone, and it is the one the data supports. It also yields the mitigation a practitioner can act on directly, in Section 6: if you cannot pin the template, do not put load-bearing instructions in a system message. For instructions that must stay in the system role for other reasons, safety and injection resistance among them, pinning the template is the remaining defence.

6 RQ3: pinning what a template renders

RQ1 counted the edits and RQ2 showed that some of them matter; RQ3 asks for a check that separates the two at build time.

289 template edits in this frame would each trip a check that hashes the template text, and about half of the decidable ones render identically. An alert channel that is half noise is muted, and a muted control is not a control.

TEMPLATELOCK pins a model by two values: the SHA-256 of its template, and a *render signature*, the hash of what that template produces on the six probe conversations. A build that finds a new template hash but the same render signature reports a cosmetic change and passes. A build that finds a new render signature reports a behavioural change and fails, because what the model is shown has changed and the prompt evaluations no longer describe the deployed system. A probe that fails to render contributes its error to the signature, so an edit that breaks a probe that previously rendered also fails the build, and a pair whose two revisions fail with different errors would alert as well. The replay below excludes the 16 events where neither revision renders from the alert accounting, tallying them separately, neither raised nor suppressed. The signature is defined relative to a pinned rendering stack, so a team upgrading its templating library re-pins at the upgrade, deliberately rather than silently.

The probe set is a parameter rather than a constant. A team whose traffic exercises serialisation paths our six probes do not, tool calls above all, can append its own probe conversations, and the render signature then covers exactly the shapes that team depends on. A probe is one fixed message list, so extending the set costs milliseconds.

Replayed over the frame's histories, a plain template hash raises 289 alerts; TEMPLATELOCK raises 137 and holds back 136 (49.8% of decidable events) as cosmetic. A pipeline that pinned at a model's first template and never re-pinned would have ended on a rendering different from the one it validated against in 99 repositories, first diverging after a median of 1.7 days.

How does this differ from pinning a commit SHA? A team that pins its model repository to a commit, which Hub tooling supports but does not do by default, prevents silent drift entirely, and a team that can freeze its dependency should. What a SHA pin cannot do is classify: it blocks the cosmetic majority of edits along with the behavioural ones, so it either freezes the pipeline out of upstream fixes or, when the pin is eventually moved, says nothing about whether the jump matters. TEMPLATELOCK is the complement rather than the rival: it tells a team when moving the pin is safe.

The cost is the reason to adopt it rather than discuss it: 9.2 ms per model for six renders and a hash, on top of a tokenizer load a build already performs. That is one wall-clock measurement on one machine and is the only number here that does not reproduce exactly; what transfers is the order of magnitude, milliseconds against a network fetch.

7 Threats to validity

The probe set bounds what we can call behavioural. Six conversations cannot exercise every path through a Jinja template, and they are six conversations rather than six independent shapes: two of them, the bare coding request and the reply-with-one-word probe,

are each a single user turn with the generation prompt requested, differing only in content, and they fire on the same number of pairs. Tool-call and structured-output serialisation, already a large part of what templates encoded at the time of this study, are not probed at all, and neither are the reasoning segments some model families encode. A template edit touching only those paths is scored cosmetic here. Every one of these gaps biases the behavioural rate *downward*, so 50.2% is a lower bound on the fraction of decidable edits that change what the model is shown, not an estimate of the fraction that changes behaviour. `TEMPLATELOCK` is also not an adversarial defence: its probes are fixed and public, so an edit crafted to render them identically while changing other paths would pass it by construction.

Weight-silence depends on recognising a weight file. The classifier keys on file extension, so a weight shipped under an unrecognised one would be counted as silence. We checked rather than assumed: across all weight-silent intervals in the frame, the extensions that do change are configuration, documentation and template files, and *no* model-weight extension appears in any of them. Extending the list to further formats leaves every figure in this section unchanged.

Rendering divergence is necessary, not sufficient. A rendering can differ by a single whitespace character and leave the model’s output untouched. This is why RQ2 exists and why it is read against a replication floor: RQ1 alone would establish only that the input changed.

The checkpoints are quantised, and one repository is a mirror. Local checkpoints are quantised rather than the Hub’s full-precision weights. Quantisation is held fixed across arms, so it cannot confound a within-checkpoint comparison, but the magnitude of an effect on full-precision weights may differ. The official Llama-3 repository is gated and cannot be read anonymously, so its template history is taken from a public mirror of the same checkpoint (NousResearch/Meta-Llama-3-8B-Instruct), whose commit history records the mirror’s uploads rather than the upstream repository’s edits.

Repair is one task family, and the probe set is six conversations. RQ2 measures single-function program repair in Python, and the cosmetic/behavioural split is decided on six fixed probe conversations. Every behavioural claim here should be read as established on that basis and not beyond it: we have not tested tool calling, retrieval-augmented prompts, long multi-turn sessions, or any non-Python domain, and a template edit that damages one of those would be invisible to this study. In a domain without an execution oracle, summarisation or open-ended reasoning, the same mechanism would surface differently. A dropped system role discards whatever the system message carried, a style constraint or a safety instruction rather than an output contract, and the degradation would be visible only to a judge, human or model, rather than to a test suite.

The one surviving resolution effect is a rendering failure, not a degraded model. Mistral-7B-Instruct-v0.2’s earlier template refuses the conversation, so its resolution difference measures a request that could not be sent rather than a model answering worse. Among comparisons where the model did receive a prompt, no resolution

difference survives correction. We report task consequence as not established rather than as absent: the uncorrected bootstrap intervals for the two Phi-3-mini-4k-instruct transitions (earlier arm minus later, 5.0 to 20.0 and -21.7 to -5.0 resolution points) lie on one side of zero, so the corrected nulls reflect the tests’ resolving power at these discordance counts, not evidence of no effect. The evidence that a template edit changes behaviour on valid input rests on output-contract compliance, one of the properties a prompt suite asserts, and not on task outcome. That evidence comes from one repository of the 5 we ran, so it establishes that the effect occurs and not how often it does: on valid input this half of RQ2 is a case study, and we do not read a rate from it.

Popularity-ranked frames are not random samples. The frame is the head of the download distribution, where maintainers are most active and most scrutinised. The out-of-frame extension suggests a lower rate further down, and we report it separately rather than pooling. Download rank also measures pulls rather than production dependence: 30 of the 400 repositories are recognisable test fixtures (tiny, dummy or random checkpoints). Excluding them leaves the drift rate unchanged, moves the settled figure from 8.5% to 6.5%, and moves the behavioural share of decidable events from 50.2% to 47.2%. Follow-up is also heterogeneous: a repository whose first template is younger than 30 days cannot yet enter the settled numerator while it stays in the denominator, so the settled figure is, if anything, understated.

We make no claim that maintainers acted wrongly. Many of these edits are corrections, and correcting a broken template is good practice. The finding is not that the edits are bad. It is that consumers cannot see them happen.

8 Related work

Prompts as evolving software artifacts have been studied by mining repositories: an analysis of 1,262 prompt changes across 243 projects finds prompt edits poorly documented, with only 21.9% explained in commit messages [12]. That work studies the prompt text an application owns. We study the artifact between that text and the model, which the application does not own and usually cannot see.

Vibe coding has so far been characterised qualitatively: a grey-literature review of 101 practitioner sources [4] and a survey of 162 participants [3] both describe a practice in which output is judged by running it and quality assurance is often skipped. Those studies establish that the verification loop is thin. We measure a dependency that such a loop is structurally unable to see.

Dependency drift is old ground in software engineering. In npm, around 12% of dependent packages were affected by breaking changes during non-major updates, and 44% of manifesting breaking changes arrived in minor and patch releases [13]. The chat-template case is the same hazard with the versioning removed. Documentation-side artifacts such as model cards [7] describe a model for humans; the chat template configures it for machines, and only the latter changes what is computed.

That a model’s output is sensitive to formatting is established rather than novel, and it is what makes this dependency worth versioning. Semantically neutral formatting changes move accuracy by up to 76 points on one open model [11]. Hosted services

drift behaviourally between releases without announcing what changed [2]. Our contribution is not that format matters; it is that on open weights the formatting artifact is a file in the repository, it is edited in place with no version, and both the edits and their behavioural consequences can therefore be measured directly rather than inferred from a black box.

Work on repair loops asks what feedback to put into the next turn [6, 9, 10]. That literature holds the serialisation fixed and varies the content. We hold the content fixed and vary the serialisation.

9 Conclusion

A prompt pipeline that versions its prompts and pins its model still depends on an executable artifact it does not version and cannot see. In 400 of the most-downloaded open-weight chat repositories, 43.8% edited that artifact after release, half of the decidable edits changed what the model is shown, and 80.6% of edits happened while no weight file moved. Much of that raw rate is launch-week churn; the figure to carry is that 8.5% of the frame (6.5% excluding the test fixtures disclosed above) changed what the model is shown more than 30 days after first publishing a template. Running 5 checkpoints under their own historical templates, 3 of 18 informative comparisons move output-contract compliance after correction; the two on valid input are the case-study repository's, and the third is the render-refusal pair, which is also the only comparison that moves task resolution. On valid input we find no surviving resolution effect and say so. Others drift with no measurable effect, which is the difficulty rather than a reassurance: from outside the repository the two look identical. A pre-registered control locates the mechanism in the system role, since moving the contract into the user turn removes both failures, which gives practitioners something to do about it besides worry. Pinning what a template renders, rather than what it says, separates the edits that matter from the ones that do not at a cost of 9.2 ms per model. Prompt CI cannot claim to pin the model's input until it versions the template too.

Data availability

The collector, the renderer, the full template histories of the frame, the execution episode log, the analysis code and TEMPLATELOCK are released at <https://github.com/s1mar/chat-template-drift> (release tag cr-1.0). The package reproduces every deterministic number

in this paper from the recorded histories, without network access; the single wall-clock cost figure transfers in order of magnitude only.

References

- [1] Matthew Carrigan. 2023. Chat Templates: An End to the Silent Performance Killer. Hugging Face blog. <https://huggingface.co/blog/chat-templates>
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. How Is ChatGPT's Behavior Changing Over Time? *Harvard Data Science Review* 6, 2 (2024). doi:10.1162/99608f92.5317da47
- [3] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2026. From Prompting to Verification: How Experience Shapes Vibe Coding Practices. *arXiv preprint arXiv:2605.24521* (2026). <https://arxiv.org/abs/2605.24521>
- [4] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2026. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. Rio de Janeiro, Brazil, 212–223. doi:10.1145/3786583.3786866
- [5] Hugging Face. 2026. Chat Templates. Transformers documentation. Accessed 2026-08-05; page resolved to Transformers v5.14.0. https://huggingface.co/docs/transformers/en/chat_templating
- [6] Mehmet Iscan. 2026. Falsification, Not Exposure: An Internally Preregistered Placebo-Controlled Decomposition of Self-Repair Feedback in Frozen Small Code Models. *arXiv preprint arXiv:2606.31511* (2026). <https://arxiv.org/abs/2606.31511>
- [7] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*. 220–229. doi:10.1145/3287560.3287596
- [8] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2024. OctoPack: Instruction Tuning Code Large Language Models. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=mw1PWNSWZP>
- [9] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=y0GJXRungR>
- [10] Jaideep Ray and Ankit Goyal. 2026. Structured Feedback Improves Repair in an LLM Agent Loop. *arXiv preprint arXiv:2607.14167* (2026). <https://arxiv.org/abs/2607.14167>
- [11] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=RIu5lyNXJT>
- [12] Mahan Tafreshipour, Aaron Imani, Eric Huang, Eduardo Almeida, Thomas Zimmermann, and Iftikhar Ahmed. 2025. Prompting in the Wild: An Empirical Study of Prompt Evolution in Software Repositories. In *Proceedings of the IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 686–698. doi:10.1109/MSR66628.2025.00106
- [13] Daniel Venturini, Filipe Roseiro Cogo, Ivanilton Polato, Marco A Gerosa, and Igor Scalante Wiese. 2023. I Depended on You and You Broke Me: An Empirical Study of Manifesting Breaking Changes in Client Packages. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–26. doi:10.1145/3576037